



Heavily based on material from:
Murach's PostgreSQL (3rd Edition), by Joel
Murach

Why Stored Procedures?

1. Stored procedures are fast.

PostgreSQL server takes some advantage of caching, just as prepared statements do.

The main speed gain comes from reduction of network traffic.

If you have a repetitive task that requires checking, looping, multiple statements, and no user interaction, do it with a single call to a procedure that's stored on the server.

2. Stored procedures are portable.

When you write your stored procedure in SQL, you know that it will run on every platform that PostgreSQL runs on, without obliging you to install an additional runtime-environment package, or set permissions for program execution in the operating system, or deploy different packages if you have different computer types.

That's the advantage of writing in SQL rather than in an external language like Java or C or PHP.

3. Stored procedures are always available as *source code* in the database itself.

It makes sense to link the data with the processes that operate on the data.

Different types of Parameters in PostgreSQL

Procedure with no parameters:

A procedure without parameters does not take any input or casts an output indirectly. It is simply called with its procedure name followed by (). It is used for simple queries.

Procedure with **IN** parameter:

An **IN** parameter is used to take a parameter as input such as an attribute. When we define an **IN** parameter in a procedure, the calling program has to pass an argument to the stored procedure. The procedure only works on the copy of the **IN** parameter. **If omitted, the default is IN.**

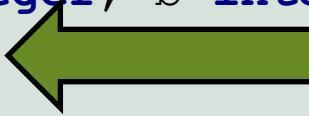
Procedure with **OUT** parameter:

An **OUT** parameter is used to pass a parameter as output or display like the select operator, but implicitly. The value of an **OUT** parameter can be changed inside the procedure and its new value is passed back to the calling program. A procedure cannot access the initial value of the **OUT** parameter when it starts.

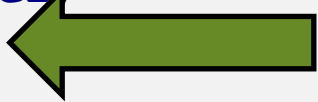
Procedure with **INOUT** parameter:

An **INOUT** parameter is a combination of **IN** and **OUT** parameters. It means that the calling program may pass the argument, and the stored procedure can modify the **INOUT** parameter and pass the new value back to the calling program.

```
create table if not exists sproc.tbl1 (  
    value int primary key  
);  
  
CREATE or replace PROCEDURE sproc.insert_data1(  
    a integer, b integer)  
LANGUAGE SQL  
AS $$  
    INSERT INTO sproc.tbl1 VALUES (a);  
    INSERT INTO sproc.tbl1 VALUES (b);  
$$;  
  
call sproc.insert_data1(1, 2);  
call sproc.insert_data1(1, 2); -- this will fail
```

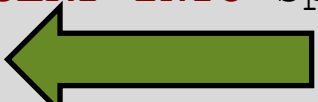


```
CREATE or replace PROCEDURE sproc.insert_data2 (a
integer, b integer)
LANGUAGE plpgsql
AS $$
    INSERT INTO sproc.tb11 VALUES (a) ;
    INSERT INTO sproc.tb11 VALUES (b) ;
$$;
```



Does not load.

```
CREATE or replace PROCEDURE sproc.insert_data2 (a
integer, b integer)
LANGUAGE plpgsql
AS $$
begin
    INSERT INTO sproc.tb11 VALUES (a) ;
    INSERT INTO sproc.tb11 VALUES (b) ;
end
$$;
```



Does load.

```
CREATE or replace PROCEDURE sproc.insert_data3(a int, b int)
LANGUAGE plpgsql
AS $$
begin
    --begin -- transaction -- syntax error
    --start transaction;
        -- compiles, but will not run
    INSERT INTO sproc.tbl1 VALUES (a, 6);
    INSERT INTO sproc.tbl2 (pk, fk) VALUES (b, a);
    INSERT INTO sproc.tbl2 (pk, fk) VALUES (b + 1, a);
    exception when others then
        raise INFO 'Error Name:%', sqlerrm;
        raise INFO 'Error State:%', sqlstate;
    --rollback; -- unnecessary
    -- return; -- unnecessary
    --commit; -- unnecessary
--end;
end
$$;
call sproc.insert_data3(1, 1);
call sproc.insert_data3(2, 3);
```

create or replace function

ap.get_vendor_id(vendor_name_param **text**)

returns integer

language plpgsql

as \$\$

declare

vendor_id_var **int** = 0;

begin

select vendor_id

into vendor_id_var

from ap.vendors

where vendor_name = vendor_name_param;

return vendor_id_var;

end; \$\$

select *

from ap.get_vendor_id('US Postal Service');

This stored procedure returns a table.

```
create or replace function ap.get_invoice(inum int)
returns table (invid int, lineitem int, invnum varchar(50))
language plpgsql
as $$
begin
    return query
        select i.vendor_id, li.invoice_sequence
            , i.invoice_number
        from ap.invoices as i
            join ap.invoice_line_items as li
            on i.invoice_id = li.invoice_id
        where i.invoice_id = inum;
end; $$;

select *
from ap.get_invoice(139);
```